

5

PART 1: FOUNDATION

Bringing the specification together

How do the first four layers combine into a specification AI can implement directly?

Four documents exist for the Riverside Clinics booking feature by the time this chapter opens, and each one is good. The vision, written with the front desk lead and a nurse in the room, names the patients who never book at all rather than say the reason out loud. The requirement from the workshop pins the booking horizon down to ninety days, a number a test can check. The Schedule Appointment use case runs the whole interaction start to finish, alternative flows included. The domain model gives Patient, Doctor, Clinic, and Appointment their attributes and the rules connecting them. Written separately, reviewed separately, each one earned its place in the last three chapters.

■ CHAPTER PROMISE

By the end of this chapter, you can assemble a vision, a requirement, a use case, and a domain model into one specification document, cross-checked so the same term means the same thing in all four places, ready for an AI assistant to implement without guessing at the connections between them.

Reading time: 12 minutes

The cost of leaving four documents apart

They also live in four different places. The vision is a paragraph from a stakeholder meeting, still sitting in someone's notes. The requirement is a line item in a tracker. The use case is a document a business analyst wrote and a developer skimmed. The domain model is a diagram an architect drew on a whiteboard and photographed. When a developer sits down to implement the feature, she opens the AI assistant and pastes in the use case, because that's the document describing what the code has to do. She doesn't attach the domain model; it lives in a different tool. She doesn't attach the ninety-day requirement either; she has seen it before and assumes the use case already covers it. On that one she is right, because the use case carries the ninety-day rule in its business rules.

The code that comes back handles the happy path correctly and misses the fact that made this feature worth building in the first place: a Riverside doctor works at more than one clinic, so a booking has to name a location that doctor actually works. Nothing in the use case she pasted said that. It lived one document over, in the domain model she never attached.

The mistake in the last three chapters was missing information: no stated reason for the system, no requirement precise enough to test, no model of what a doctor and a patient are to the business. This chapter's mistake is different. Every piece of information exists. It simply isn't in the same room.

An AI assistant, like a new hire, implements from what it is handed. Hand it a complete use case and nothing else, and it will implement that use case correctly, on its own terms. It has no way of knowing that a business rule belonging to the use case lives in a domain model two documents away, because from where it sits, that document does not exist. The resulting code follows the instructions it was given, to the letter. Those instructions were incomplete because nobody had put the specification back together before handing it over, even though every piece of it already existed.

The review that follows looks identical to the one in Chapter 1: a developer finds the gap, traces it back, and either patches the code or goes looking for the missing piece. The hours lost are the same hours. The cause is different enough that most teams never connect the two problems, because the second one hides behind documents that all, individually, look finished.

One document, four questions answered

Each layer built in Chapters 2 through 4 answers a question the others don't. The vision answers why the system exists and for whom. The requirement answers what specific, observable behavior it must have. The use case answers what a complete interaction looks like, start to finish. The domain model answers what business objects the system runs on, and how they relate. None of the four can stand in for another. A vision that lists database fields has stopped being a vision. A domain model that describes user steps has stopped being a domain model.

A complete specification places its four documents in agreement with each other, in one place, before anyone, human or AI, starts implementing from any single one of them. Agreement here means something specific: the same term names the same thing everywhere it appears. If the vision names a rule about doctors and locations, that rule has to survive into the requirement as something testable, into the use case as a step or a business rule, and into the domain model as a relationship between two entities. A rule that appears in only one of the four layers is a rule your AI assistant may never see, because what it reads is what it was handed.



A specification is complete when its four layers agree with each other in one place, not when each layer merely exists.

Assembling the four layers into one document

- ① Place the vision at the top, in the words the stakeholders agreed to, not restated by whoever is assembling the document. A reader should meet the reason before the rule.
- ② Attach only the requirements this feature's use case depends on, not the full backlog. A ninety-day booking window belongs here. An unrelated requirement about billing does not.
- ③ Add the use case in full: actor, preconditions, main scenario, alternative flows, postconditions, business rules. This is the shape an AI assistant implements best, and summarizing it defeats the reason for including it.
- ④ Add the domain model, limited to the entities and attributes the use case names. Then check each one against the use case by name; a use case that says "Doctor" and a model that says "Physician" are describing the same thing to a person and two different things to an AI assistant.
- ⑤ Version the whole assembly as one document, dated. When a requirement changes, this is the one file that changes, not four scattered guesses reconciled after the fact.

The Riverside Clinics specification, assembled

Here is what steps one and two produce for Riverside Clinics: the vision from Chapter 2, condensed, with the multi-location fact from Chapter 4's model added to it, followed by the one requirement this use case depends on most directly, the rule that turns "available" into something a system can check rather than something a receptionist remembers.

Specification: Schedule Appointment, Riverside Clinics (v1.0)

1. VISION

Patients book by phone or in person at the front desk. Which doctor is at which location on which day is knowledge only the front desk holds, so a patient booked at the wrong site arrives and is turned away. Both routes also require stating the reason for a visit where staff, and sometimes other patients, can hear it. Some patients tolerate that. Others avoid booking entirely rather than disclose it aloud, and the clinic never learns they needed care. We want patients to book, reschedule, and cancel appointments online, privately. This system is not responsible for billing, medical records, or clinical intake, which stay with the systems that already handle them. Chapter 4's model then recorded in writing what the vision had left as front desk knowledge: a doctor works at one or more clinics.

2. REQUIREMENT (REQ-07: availability)

A time slot is available for booking only if all three hold: it falls within 90 days of the current date; the doctor works at that location; and no existing appointment already occupies it. The system must not display a slot to a patient unless all three are true, and must state which condition failed when a patient requests one that isn't.

The specification, continued

The use case below is the one Chapter 3 wrote, condensed. In the working document it sits at full length, exactly as Chapter 3 has it, every step and every alternative flow written out.

Specification: Schedule Appointment, Riverside Clinics (continued)

3. USE CASE: Schedule Appointment

Actor: registered patient, or a receptionist booking on the patient's behalf.
Preconditions: patient is logged in; at least one doctor is accepting bookings within the request window.

Main scenario: (1) patient selects a location or a doctor; (2) system displays only slots satisfying REQ-07; (3) patient selects a slot; (4) system reserves the slot and creates the appointment record; (5) system sends confirmation naming the doctor, date, time, and location.

Alternative flows: if no slot satisfies all three conditions, the system offers the same doctor at a later date or another doctor with availability in the requested range, rather than a bare "not available." If a slot is taken between steps 2 and 4, the system notifies the patient and redispays current availability.

Postconditions: the appointment exists, tied to one patient, one doctor, one location, one time slot; confirmation has been sent.

Business rules: slots run fifteen minutes, inside clinic hours; cancellation requires at least 24 hours' notice, otherwise the patient calls the clinic; doctor availability is entered by clinic staff, not pulled from an external calendar.

The specification, concluded

Specification: Schedule Appointment, Riverside Clinics (concluded)

4. DOMAIN MODEL (entities the use case above names)

Patient: a registered account; has many Appointments. Doctor: works at one or more Clinics, many-to-many, the relationship this whole feature turns on; has many Appointments. Clinic: one physical Riverside location; employs many Doctors, handles many Appointments. Appointment: belongs to one Patient, one Doctor, one Clinic, one TimeSlot; status is Scheduled, Completed, Cancelled, or NoShow. TimeSlot (value object): date, start time, end time; has no identity of its own outside an Appointment.

Read start to finish, one fact threads through all four sections in the same words. What the vision records as a doctor working at one or more clinics becomes the second condition in REQ-07, becomes the filter the use case applies before it shows a patient anything, and becomes the many-to-many Doctor-Clinic relationship in the domain model. An AI assistant implementing from this one document doesn't have to go looking for that connection. It has already been made.

Where assembly still goes wrong

▲ WARNING

Assembling the four layers once is not the same as keeping them assembled. A specification that stops changing the day the feature ships decays the same way any other documentation decays, unnoticed until it no longer describes what is running.

Treating assembly as a one-time step

Six months after launch, a rule changes in production and nobody updates the assembled document. New work against the feature reverts to guessing, because the one place that was supposed to hold the truth no longer does.

Assembling everything instead of what one use case needs

Pulling in the full domain model and the entire requirements backlog produces a document long enough that reviewers stop reading it, which defeats the reason it was assembled in the first place.

Field guide: the one-page assembly template

Six sections, one file. The note against each says what belongs there, and nothing about how to word it.

- ☐ Header: feature name, version number, date. One line, so a reader knows which assembly they are holding and which one it replaced.
- ☐ 1. Vision: why this exists and for whom, in the words the stakeholders agreed to.
- ☐ 2. Requirements: only the ones this use case depends on, each written as something a test can check.
- ☐ 3. Use case: actor, preconditions, main scenario, alternative flows, postconditions, business rules. All six lines filled.
- ☐ 4. Domain model: the entities, attributes, and relationships the use case names, under the names the use case uses.
- ☐ 5. Sign-off: the date the four sections above were last cross-checked against each other, so a reader knows how current the agreement between them is.

Fill every section before handing the document to an AI assistant or a new team member. A section left blank is a decision someone downstream makes on your behalf, usually without knowing they made it.

What to remember

- A specification is complete when its four layers exist together and agree with each other, not when each one merely exists somewhere.
- Assembling costs time up front. That cost is repaid the first time a requirement changes, since one document gets updated instead of four that nobody reconciles.
- The assembled specification, not the code generated from it, is what should still be correct after the code has been rewritten twice.
- Consistency between layers, the same term meaning the same thing everywhere it appears, matters more than how complete any single layer is on its own.

QUESTIONS FOR YOUR TEAM

- Can anyone here point to one document containing this feature's vision, requirement, use case, and domain model together, or does it take four open tabs?
- The last time a requirement changed, did the domain model and the use case change with it, or only the code?

ONE ACTION TO TAKE NEXT

Take a feature currently in flight. Gather whatever exists for it, vision, requirements, use case, domain model, even if some of it is incomplete, and assemble it into one document today.

Cross-check it once against the field guide on the previous page before anyone implements another line against it.

TRANSITION. This assembled document tells an AI assistant what to build and why. It doesn't yet tell it how: which architecture, which layering, which naming rules apply to code that has to sit next to everything else Riverside has already built. Architecture is the layer the next chapter adds. The two chapters after it add the reusable implementation knowledge a team encodes once, and the workflow that carries a finished specification through to shipped code.

Bringing the specification together ends here.